

Programming Assignment #3

MAT 399A, FALL 2025

Due Friday, October 31

1 Quantum circuits and measurements

In the previous programming assignment you constructed a pure quantum circuit without measurement. In particular, the circuit was created using

```
<circuit> = qiskit.QuantumCircuit(<n>)
```

where $\langle n \rangle$ is the number of qubit wires to use. Applying the circuit to an initial quantum state $|\psi_0\rangle$ results in a final quantum state $|\psi\rangle = U|\psi_0\rangle$, where U is the unitary matrix that the circuit represents. However, the state $|\psi\rangle$ cannot be measured directly. We can only project onto a classical bit register.

Measured circuits

To create a quantum circuit that can be measured, we must create a circuit that has both quantum qubits and classical bits:

```
<circuit> = qiskit.QuantumCircuit(<n>,<m>)
```

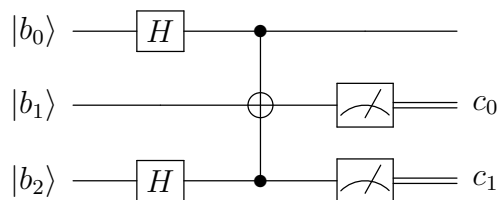
where $\langle m \rangle$ is the number of classical bits. The number of classical bits is often equal to the number of quantum qubits, but can be less. The qubits must be mapped to the bits, which is done through the function

```
<circuit>.measure(<qubit list>,<cbit list>)
```

where $\langle \text{qubit list} \rangle$ is a list of the qubit wire numbers that are to be measured, and $\langle \text{cbit list} \rangle$ is a list of the classical bit indices that form the target bit register that holds the results of the measurement. Both lists are of length m .

For example, the following measured quantum circuit has 3 quantum qubits and 2 classical bits. The classical register holds the result of measuring the qubit wires numbers 1 and 2. The qubit on wire number 0 is not measured, and is called an *ancilla*.

```
import qiskit
qc = qiskit.QuantumCircuit(3,2)
qc.h([0,2])
qc.mcx([0,2],1)
qc.measure([1,2],[0,1])
```



Note that the `qc.draw()` call will give a somewhat different graphical representation of the circuit.

Default initial state. When a quantum circuit is run, the initial state $|\psi_0\rangle$ defaults to the ground state $|0\rangle_n = |0\rangle|0\rangle\cdots|0\rangle$ (n copies of $|0\rangle$). While it is possible to specify a different initial state, we will always use the default. So that in the above example, the initial state is $|\psi_0\rangle = |b_2\rangle|b_1\rangle|b_0\rangle = |0\rangle_3$.

Simulating a circuit measurement

A measured quantum circuit can be run on a quantum computer. Typically the circuit is run multiple times, and results of the runs (“shots”) accumulated. We will not run our circuits on an actual quantum computer, but instead use the Python `qiskit_aer` package to simulate the results of an experiment. Running the simulator is done through the `run` member function of the `AerSimulator` class.

```
<run> = qiskit_aer.AerSimulator().run(<circuit>,shots=<N>)
```

where `<N>` is the number of times the circuit should be run. The experimental data can be extracted from the run via

```
<data> = <run>.result().data()['counts']
```

The left-hand side is a dictionary with keys given by the values (in hexadecimal) of the classical registers. The value stored at given key in the dictionary is the number of times a register took that particular hexadecimal value. However if a value never occurred, it is not present in the dictionary.

As an example, for a quantum circuit with two classical bits that is run 100 times, the output might be

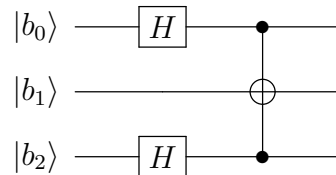
```
{'0x2': 22, '0x0': 54, '0x3': 24}
```

Of the 100 runs, 54 resulted in a (2-bit) classical register value of $00_2 = 0$, 22 resulted in a value of $10_2 = 2$, and 24 resulted in a value of $11_2 = 3$. On the other hand, the classical register value of $01_2 = 1$ never occurred.

A worked example

Consider the quantum circuit, but without the measurement, from the previous example. We can work out the corresponding unitary matrix U in the computational basis using the techniques from programming assignment #2. The result is

$$U = \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 & 0 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 & -1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & -1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & -1 & -1 \\ 1 & -1 & 0 & 0 & -1 & 1 & 0 & 0 \end{pmatrix}$$

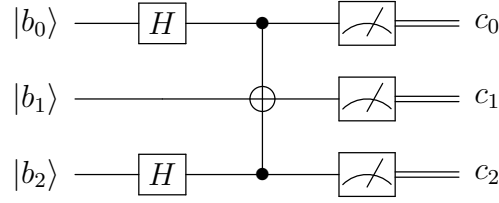


In particular, if the initial state is $|\psi_0\rangle = |000_2\rangle$, then the result of applying U to the initial state is just given by the first column of U . More explicitly, the final state is

$$|\psi\rangle = \frac{1}{2} |000_2\rangle + \frac{1}{2} |001_2\rangle + \frac{1}{2} |100_2\rangle + \frac{1}{2} |111_2\rangle. \quad (1)$$

The probability that the 3-bit classical register has a value of $(c_2c_1c_0)_2$ is given by the square of the complex modulus of the coefficient of $|c_2c_1c_0\rangle$ in $|\psi\rangle$.

So if we measure all three qubits in the circuit as in the circuit diagram,



Equation (1) tells us that the probabilities that the classical register values are measured to be 000_2 , 001_2 , 100_2 , or 111_2 are all $\frac{1}{4}$:

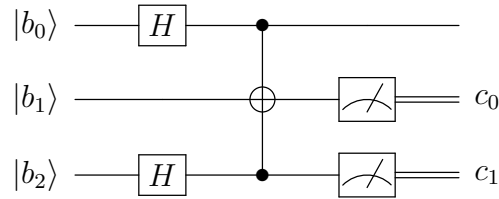
$$\text{prob}(000_2) = \frac{1}{4}, \quad \text{prob}(001_2) = \frac{1}{4}, \quad \text{prob}(100_2) = \frac{1}{4}, \quad \text{prob}(111_2) = \frac{1}{4}$$

While any other register value occurs with zero probability. It follows that for a run of (say) 100 shots, the nonzero expectation values for the frequency of occurrence of the classical register values are:

$$000_2 : 25, \quad 001_2 : 25, \quad 100_2 : 25, \quad 111_2 : 25.$$

In a real experiment, the actual measured frequencies of these register values would be close to 25, but not necessarily exactly 25.

Now suppose that we use qubit wire number 0 as an ancilla as in the measured quantum circuit below.



To get the probability that the classical register measures $(c_1c_0)_2$, we still measure all three classical bit values as before, but we drop the bit corresponding to qubit wire 0. For example, the 2-bit register value 00_2 occurs when the 3-bit register values are either 000_2 or 001_2 . Thus the probability of 00_2 occurring is $\text{prob}(000_2) + \text{prob}(001_2) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$. The probabilities of the other 2-bit register values are computed similarly. And we get

$$\text{prob}(00_2) = \frac{1}{2}, \quad \text{prob}(10_2) = \frac{1}{4}, \quad \text{prob}(11_2) = \frac{1}{4}$$

(the value 10_2 occurs with zero probability). For a run with 100 shots, the expectation values for the occurrence frequencies are consequently

$$00_2 : 50, \quad 10_2 : 25, \quad 11_2 : 25$$

Of course in a real experiment, deviation from these values would be expected.

2 Programming tasks

1. Compute the theoretical classical register values for an unmeasured quantum circuit with no ancillae.

```
def probabilities0(qc):
    """
    -- probabilities of classical register values, assuming no ancillae
    where:
        qc = Qiskit quantum circuit with no classical bits (unmeasured)
    assumes:
        qc.num_clbits == 0
    returns:
        list of register probabilities, indexed by register value
    """
```

Note that the list that is returned by the function has length 2^n , where n is the number of qubit wires used in the circuit. You will need to obtain the unitary matrix that the circuit realizes. This can be obtained using the `qiskit.quantum_info` class:

```
U = qiskit.quantum_info.Operator(qc).to_matrix()
```

(this is why you were forbidden from using this class in the previous assignment).

2. Use the Qiskit circuit simulator package to obtain the results of running a quantum circuit with no ancillae.

```
def measurements0(qc,N):
    """
    -- get experimental results of running a quantum circuit with no ancillae
    where:
        qc = Qiskit quantum circuit with no classical bits (unmeasured)
        N = number of shots
    assumes:
        qc.num_clbits == 0
    returns:
        list of measured register value frequencies, indexed by register value
    """
```

The circuit `qc` passed in to the function is assumed to be an unmeasured circuit. So you will need to construct a measured quantum circuit in order to run the simulator. The code fragment

```
n = qc.num_qubits
qcm = qiskit.QuantumCircuit(n,n)
qcm.compose(qc,range(0,n),inplace=True)
```

will create a new quantum circuit with n qubits and n classical bits that is a copy of `qc`. You will then need to map the qubits to the classical bits using the `measure` function of the `QuantumCircuit` class.

3. Compute the theoretical classical register values for an unmeasured quantum circuit with ancillae.

```
def probabilities(qc,qbits,cbits):
    """
    -- probabilities of classical register values, with ancillae
    where:
        qc      = Qiskit quantum circuit with no classical bits (unmeasured)
        qbits   = list of qubit wire numbers to measure
        cbits   = list of classical bits the qubits are mapped to
    assumes:
        qc.num_clbits == 0
        len(qbits) == len(cbits) == m
        cbits is a permutation of range(0,m)
    returns:
        list of measured register value frequencies, indexed by register value
    """
```

The two lists `qbits` and `cbits` set up the mapping from the qubit wires of the circuit to the classical register bits. This is the same mapping as would be specified in the `measure` function of the `QuantumCircuit` class — although you will not use that function here (you will use it in the next task).

This task is programming challenge! There are some hints and suggestions at the end of this handout.

4. Use the Qiskit circuit simulator package to obtain the results of running a quantum circuit with ancillae.

```
def measurements(qc,qbits,cbits,N):
    """
    -- get experimental results of running a quantum circuit with ancillae
    where:
        qc      = Qiskit quantum circuit with no classical bits (unmeasured)
        qbits   = list of qubit wire numbers to measure
        cbits   = list of classical bits qubits are mapped to
        N       = number of shots
    assumes:
        qc.num_clbits == 0
        len(qbits) == len(cbits) == m
        cbits is a permutation of range(0,m)
    returns:
        list of measured register value frequencies, indexed by register value
    """
```

What to turn in

The code for all of the above functions should be put in a single file named `exercise3.py`. You may only import the packages `numpy`, `qiskit`, `qiskit_aer`, `math`, and `cmath`.

Test driver

A simple test driver `exercise3_test.py` has been uploaded for convenience in testing. If you run

```
python exercise3_test.py
```

the output should match the contents of the file `exercise3_test.output.txt` to a reasonable extent. Note that the output of the Qiskit simulator will be different every time you run it, so your output for questions 2 and 4 is unlikely to exactly match the corresponding values given in `exercise3.output.txt`.

Hints and suggestions for question #3

An n -bit integer J has a binary representation $J = J_{n-1} \cdots J_1 J_0$. That is,

$$J = J_{n-1}2^{n-1} + \cdots J_12^1 + J_02^0.$$

Moreover, we can examine the i -th bit ($0 \leq i < n$) of J by performing a bit-wise AND with the mask 2^i :

$$J_i = \begin{cases} 1 & \text{if } J \ \& \ 2^i \neq 0 \\ 0 & \text{if } J \ \& \ 2^i = 0. \end{cases}$$

In particular, for a quantum circuit with n qubits, the computational basis vectors are of the form

$$|J\rangle_n = |J_{n-1}\rangle \cdots |J_1\rangle |J_0\rangle, \quad \text{where } 0 \leq J < 2^n.$$

And m -bit classical register values I are of the form

$$I = I_{m-1} \cdots I_1 I_0, \quad \text{with } 0 \leq I < 2^m.$$

Now let $q = (q_0, \dots, q_{m-1})$ and $c = (c_0, \dots, c_{m-1})$ denote the mapping lists `qbits` and `cbits` used in the `probabilities` function. The computational basis vector $|J\rangle_n$ maps to the classical register value I provided that the bits of J and I satisfy

$$J_{q(i)} = I_{c(i)}, \quad \text{for } 0 \leq i < m.$$

Or equivalently,

$$I = \sum_{0 \leq i < m} J_{q(i)} 2^{c(i)}.$$

And we write $I = \text{map}(J)$. For example, if there are $n = 3$ qubits and the mappings are $q = (1, 2)$ and $c = (1, 0)$, then

$$I_1 = I_{c(0)} = J_{q(0)} = J_1 \quad \text{and} \quad I_0 = I_{c(1)} = J_{q(1)} = J_2.$$

So that for $J = 101_2$, we have $I_0 = J_2 = 1$ and $I_1 = J_1 = 0$. I.e., $\text{map}(101_2) = I = I_1 I_0 = 01_2$. Note that we also have $\text{map}(100_2) = 01_2$.

To find the probability $cprob(I)$ that the classical register value I occurs as a result of measuring the final state of the circuit, we sum over all probabilities of computational basis states $|J\rangle_n$ that map to I :

$$cprob(I) = \sum_{\substack{0 \leq J < 2^n \\ I = \text{map}(J)}} qprob(J).$$

Here $qprob(J)$ denotes the probability that state $|J\rangle_n$ occurs in the final state. One possible strategy for this computation would be to, for a given I , find all states $|J\rangle_n$ that map to I . However, because we need to do this for all I , a simpler strategy is to iterate over the computational basis states $|J\rangle_n$, and accumulate the probability for classical register value I whenever $|J\rangle_n$ maps to I .

```
cprob = [0,0,...,0] # 2**m items
for J in range(0,2**n):
    I = map(J)
    cprob[I] += qprob[J]
```